18/26

SHORT ANSWER TEST

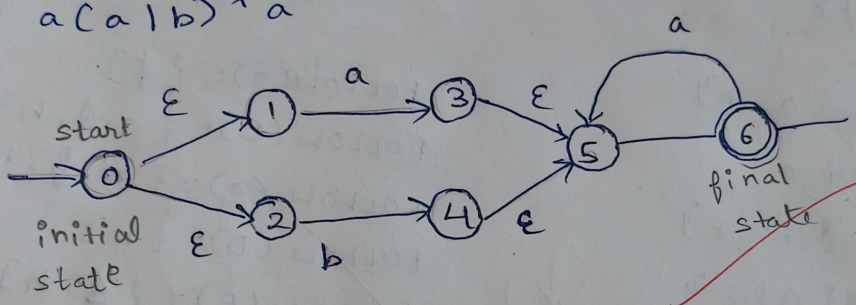
Name: Juhi Surin
Reg. no.: 192525148
Course: CSA1407
Compiler Design

54/100

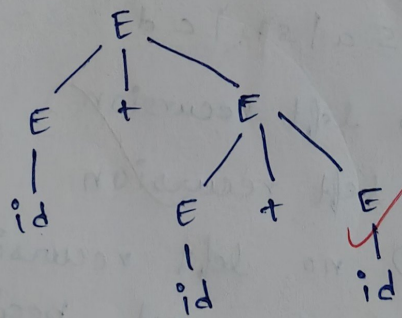
1) given grammar: $E \rightarrow E + E \mid id$ is ambigious
it can generate more than one production.

$E \rightarrow E+E \mid id$
$E \rightarrow E+E$
$E \rightarrow id$

2) a) $a(a|b)^*a$



3) parse tree for id+id*id



4) given grammar: $A \rightarrow A\alpha \mid \beta$
$A \rightarrow A\alpha$
$A \rightarrow \beta$
$A \rightarrow \beta A'$
$A' \rightarrow \alpha A' \mid \varepsilon$

5) $S \rightarrow iEtS \mid iEtSeS \mid a$

$S \rightarrow iEtS \mid a$

$S' \rightarrow eS \mid \varepsilon$

$S \rightarrow iEtSS' \mid a$

$S' \rightarrow eS \mid \varepsilon$

6) $S \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bC \mid \varepsilon$

$D \rightarrow EF$

$E \rightarrow g \mid \varepsilon$

$F \rightarrow f \mid \varepsilon$

$\alpha B \beta$

FIRST(S) = { a }

FIRST(B) = { c }

FIRST(C) = { b, ε }

FIRST(D) = { g, f, ε }

FIRST(E) = { g, ε }

FIRST(F) = { f, ε }

FOLLOW(S) = { $ }

FOLLOW(B) = { g, f, h }

FOLLOW(C) = { g, f, h }

FOLLOW(D) = { h }

FOLLOW(E) = { f, h }

FOLLOW(F) = { g, h }

7) given grammar : $S \rightarrow Sa \mid Sb \mid cd$

$S \rightarrow Sa$ ✓ ⇒ left recursion

$S \rightarrow Sb$ ✓ ⇒ left recursion

$S \rightarrow cd$ ✗ ⇒ no left recursion because it does not begin with same non-terminal.

So it contains immediate left recursion. To remove this we should do LL(1) parsing.

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

FIRST(E') = {+, ε}

9) $S \rightarrow (S)S \mid \varepsilon$

((()))

It is valid.

10) Shift - Reduce Parsing :-

i) Shift → move the next input into the stack.

ii) Reduce → replace the symbols in the stack with non-terminal

iii) Accept → The input string is correct and parsing is complete.

iv) Error → The input string is wrong then it cannot be parsed then redo the step.

11) grammar : $S' \rightarrow S$

$S \rightarrow CC$

$C \rightarrow cC \mid d$

item : $[S' \rightarrow \cdot S]$ list all LR(0) state

closure ($[S' \rightarrow \cdot S]$) ?

$S' \rightarrow \cdot S$

$S \rightarrow CC$

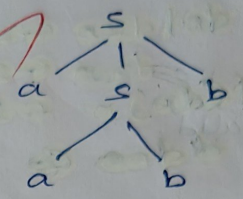
$C \rightarrow cC \mid d$

12)

$S \to AB$
$A \to a$
$B \to b$

FIRST (S) = {a}
FIRST (A) = {a}
FIRST (B) = {b}

for $S \to asb|ab$
aabb is accepted



13)

$S \to A$
$A \to aB|Ad$
$B \to b$
$C \to g$

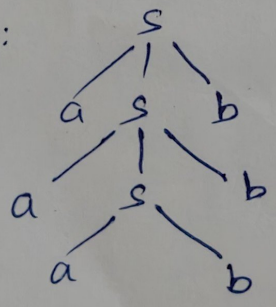
FIRST (S) = {a}
FIRST (A) = {a}
FIRST (B) = {b}
FIRST (C) = {g}

FOLLOW (S) = {$}
FOLLOW (A) = {d, $}
FOLLOW (B) = {$}
FOLLOW (C) = {.}

15) given grammar:

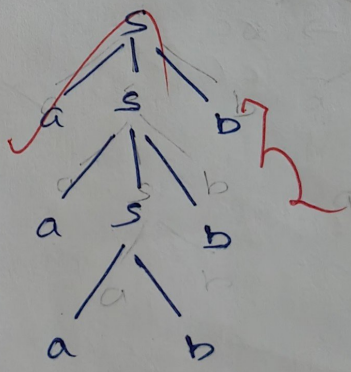
$S \to AB$
$A \to aA|a$
$B \to bB|b$

RMD:



LMD:

aaabbb



S → AB
A → aA | ε
B → bB | c

FIRST(S) = {a, b, c}
FIRST(A) = {a, ε}
FIRST(B) = {b, c}

FOLLOW(S) = {$}
FOLLOW(A) = {b, c}
FOLLOW(B) = {$}

3

17) ★ Eliminate Left Recursion

A grammer is said to be left recursive if a non-terminal is the first symbol in right hand side of the production.

i) first identify the production it has to be in the form of $A \rightarrow A\alpha | \beta$

ii) if A is not present then introduce A' with β.

iii) Replace A with A' in the given production and include ε at the last.

$A \rightarrow A\alpha | \beta$

$A \rightarrow A\alpha$

$A \rightarrow \beta$

$A \rightarrow A'\beta$

$A' \rightarrow A'\alpha | \varepsilon$

3

★ Left Factoring

i) Find common prefix in the production.

ii) introduce a non-terminal.

iii) factor out the common prefix. from there.

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 | \beta_2$

18) grammar :

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

Input string : id + id * id

i) Recursive - Descent (Top-Down) Parsing

ii) shift Reduce (Bottom up

i) Recursive - Descent (Top-Down) Parsing :

⇒ Left recursion

$E \to TE'$

$E' \to +TE' \mid \varepsilon$

$T \to FT'$

$T' \to *FT' \mid \varepsilon$

$F \to id$

⇒ FIRST() & FOLLOW()

FIRST (E) = { id }

FIRST (E') = { +, ε }

FIRST (T) = { id }

FIRST (T') = { *, ε }

FIRST (F) = { id }

FOLLOW (E) = { $ }

FOLLOW (E') = { $ }

FOLLOW (T) = { +, $ }

FOLLOW (T') = { +, $ }

FOLLOW (F) = { * }

⇒ predictive parsing

Non-terminal	id	+	*	(	)	$
E	$E \to TE'$					
E'		$E' \to +TE'$				$E' \to \varepsilon$
T	$T \to FT'$					
T'		$T' \to \varepsilon$	$T' \to *FT'$			$T' \to \varepsilon$
F	$F \to id$					

⇒ stack implementation

symbol	stack	o/p
$	id + id * id $	E → TE'
$E'T	id + id * id $	T → FT'
$E'T'F	id + id * id $	F → id
$E'T'id	id + id * id $	~~T' → ε~~
$E'T'	+ id * id $	T' → ε
$E'	+ id * id $	E' → +TE'
$E'T+	+ id * id $	
$E'T	id * id $	
$E'T'F	id * id $	T → FT'
$E'T'id	id * id $	F → id
$E'T'	* id $	
$E'T'F*	* id $	T' → ε
$E'T'id	id $	F → id
$E'T'	$	=
$E'	$	T' → ε
$	$	E' → ε
	$	Accept

ii) Shift Reduce Parsing

⇒ Left recursion

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow id$

⇒ FIRST( ) & FOLLOW ( )

FIRST(E) = { id }

FIRST(E') = { +, ε }

FIRST(T) = { id }

FIRST(T') = { *, ε }

FIRST(F) = { id }

FOLLOW(E) = { $ }

FOLLOW(E') = { $ }

FOLLOW(T) = { +, $ }

FOLLOW(T') = { +, $ }

FOLLOW(F) = { * }

⇒ predictive parsing

Non-terminal	id	+	*	(	)	$
E	$E \rightarrow TE'$					
E'		$E' \rightarrow +TE'$				$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$					
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow F*T'$			$T' \rightarrow \varepsilon$
F	$F \rightarrow id$					

10

19) given grammar:

$E \to E + E$

$E \to E * E$

$E \to (E)$

shift reduce parsing

input string : id + id * id

~~$E \to E + E$~~

~~$E \to E * E$~~

~~$E \to (E)$~~

~~$E \to id$~~

3)

4)

10)
```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    int sum = a+b;
    printf ("%d", sum);
    return 0;
}
```

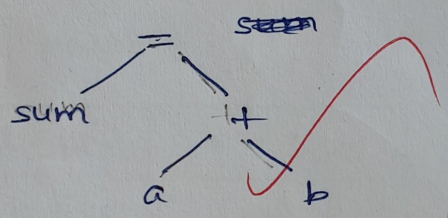
Phases of compiler

i) Lexical Analyser:

Lexemes: int, main, (,), {, a, =, 10, b, 20, ;, sum, =, +, printf, (, ", %, d, return, 0, ;.

ii) Syntax Analyzer:

sum = a + b



iii) semantic analyzer:

int a = 10 ✓

int b = 20 ✓

int sum = a + b ✓

hence it is semantically correct

iv) intermediate code generator:

$t_1 = 10$;

$t_2 = 20$;

$t_3 = t_1 + t_2$;

$t_4 = t_3$;

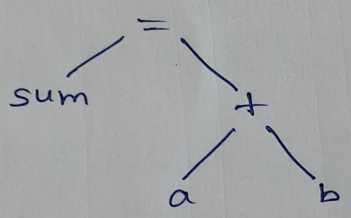
code optimizer :

```
t1 = 10;
t2 = 20;
t3 = t1 + t2;
```

vi) code generator :

```
MOV  R0, a
MOV  R1, b
ADD  R0, R1
STR  R0
```

c) Parse tree



b) sum = a + b